Question 1: Simple Interest Calculation using Maven and Azure Pipeline

Develop a Java application to calculate simple interest based on the principal amount, rate of interest, and time period provided as input.

- Build the application as a Maven project.
- Configure an Azure Pipeline to compile the source code.
- Generate the JAR artifact.

Question 2: Student Pass/Fail using JUnit and Azure Pipeline

Develop a Java application to determine whether a student has passed or failed based on the marks obtained.

- Pass: Marks ≥ 50
- Fail: Marks < 50
- Write JUnit test cases for marks 35, 50, 75, and 100.
- Configure an Azure Pipeline using Maven to build the application and run regression tests.

Question 3: Gross Salary Calculation using Jenkins

Develop a Java application to calculate and display the gross salary of an employee using the formula:

Gross Salary = Basic Salary + HRA + DA

- Accept basic salary, HRA, and DA as input.
- Install and configure Jenkins on a cloud virtual machine.
- Configure Java.
- Create a Jenkins job to compile and execute the application.

Question 4: Palindrome Number using Jenkins CI

Develop a Java application to determine whether a given integer reads the same forwards and backwards.

- Configure a Jenkins CI pipeline.
- Retrieve the source code from GitHub.
- Compile the program.
- Execute the application.

Question 5: Apache Web Server Deployment using Ansible

Create an Ansible playbook to:

- Install the Apache web server on a Linux system.
- Start and enable the Apache service.
- Deploy a simple HTML webpage displaying a welcome message.
- Verify the application's availability using an HTTP request.

Question 6: Vowel or Consonant using Gradle

Develop a Java application to read an English alphabet character and determine whether it is a vowel or a consonant.

- Create a Gradle Java project.
- Configure the build script.
- Compile and package the application into a JAR file.

Question 1: Simple Interest Calculation using Maven and Azure Pipeline

1. Aim

To develop a Java application to calculate simple interest using the principal amount, rate of interest, and time period provided as input, build the application using Maven, and configure an Azure Pipeline to compile the source code and generate a JAR artifact.

2. Formula

$$SI = \frac{P \times R \times T}{100}$$

Where:

- P – Principal amount
- R – Rate of interest per annum
- T – Time period in years
- SI – Simple interest

$$Amount = P + SI$$

3. Requirements

- Ubuntu or Windows operating system
- Java JDK 17
- Apache Maven
- Git

- GitHub account
- Microsoft Azure DevOps account

4. Procedure

Step 1: Check Java installation

Open the terminal and execute:

```
None
java -version
```

Check the Java compiler:

```
None
javac -version
```

If Java 17 is not installed on Ubuntu:

```
None
sudo apt update
sudo apt install openjdk-17-jdk -y
```

Verify the installation:

```
None
java -version
```

Step 2: Install Maven

Execute:

```
None
sudo apt install maven -y
```

Verify Maven:

```
None
mvn -version
```

The output should display the Maven version and the Java version used by Maven.

Explanation: Maven is a build automation tool used to compile Java source code, manage dependencies, execute tests, and package applications into JAR files.

Step 3: Create a Maven project

Create a project directory:

```
None
mkdir SimpleInterest
cd SimpleInterest
```

Generate a Maven project using the quickstart archetype:

```
None
mvn archetype:generate \
-DgroupId=com.example \
-DartifactId=SimpleInterest \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

Navigate into the generated project:

```
None
cd SimpleInterest
```

The generated project structure will be:

```
None
SimpleInterest/
├── pom.xml
└── src/
    ├── main/
    │   └── java/
    │       └── com/
    │           └── example/
    │               └── App.java
    └── test/
        └── java/
            └── com/
```

```
                └── example/
                    └── AppTest.java
```

Explanation:

- `pom.xml` contains the Maven project configuration.
- `src/main/java` contains the Java application.
- `src/test/java` contains test programs.
- `App.java` is the main Java application.

Step 4: Write the Java program

Open the Java source file:

```
None
nano src/main/java/com/example/App.java
```

Replace the existing code with the following:

```
None

package com.example;

import java.util.Scanner;

public class App {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Principal Amount: ");
        double principal = sc.nextDouble();

        System.out.print("Enter Rate of Interest: ");
        double rate = sc.nextDouble();

        System.out.print("Enter Time Period (in years): ");
        double time = sc.nextDouble();
```

```
        double simpleInterest =
                (principal * rate * time) / 100;

        double totalAmount = principal + simpleInterest;

        System.out.println("Simple Interest: " + simpleInterest);
        System.out.println("Total Amount: " + totalAmount);

        sc.close();
    }
}
```

Save the file by pressing `Ctrl + O`, Enter, and `Ctrl + X`.

Explanation of the program:

1. The `Scanner` class accepts input from the user.
2. The principal amount, interest rate, and time period are stored in variables.
3. The simple interest is calculated using the formula.
4. The total amount is calculated by adding the principal and simple interest.
5. The calculated values are displayed using `System.out.println()`.

Step 5: Configure the Maven build file

Open the `pom.xml` file:

```
None
nano pom.xmlnano
```

Replace its contents with:

```
None

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.example</groupId>
    <artifactId>SimpleInterest</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding
>
    </properties>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive>
                        <manifest>

<mainClass>com.example.App</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

Save and exit.

Explanation:

- `groupId` identifies the project group.
- `artifactId` specifies the project name.
- `version` defines the project version.
- `maven.compiler.release` specifies Java 17.
- `maven-compiler-plugin` compiles the Java source code.
- `maven-jar-plugin` packages the application into a JAR file.
- `mainClass` specifies the entry point of the application.

Step 6: Compile and execute the Maven application

Compile the application:

```
None
mvn compile
```

Expected output:

```
None
BUILD SUCCESS
```

Run the application using Maven:

```
None
mvn exec:java -Dexec.mainClass="com.example.App"
```

If the exec plugin is not configured, use the following command to run the packaged JAR after Step 7.

Example input:

```
None
Enter Principal Amount: 10000
Enter Rate of Interest: 5
Enter Time Period (in years): 2
```

Expected output:

```
None
Simple Interest: 1000.0
```

```
Total Amount: 11000.0
```

Step 7: Generate the JAR artifact

Execute:

```
None
mvn clean package
```

Explanation of the command:

- `clean` removes previously generated build files.
- `package` compiles the application and packages it into a JAR file.

The JAR file will be generated inside the `target` directory.

Check the generated file:

```
None
ls target
```

Expected output includes:

```
None
SimpleInterest-1.0-SNAPSHOT.jar
```

Run the JAR file:

```
None
java -jar target/SimpleInterest-1.0-SNAPSHOT.jar
```

Enter the principal, rate, and time period when prompted.

This confirms that the application has been packaged and can run independently.

5. Configure Azure Pipeline

Step 8: Create a GitHub repository

1. Open GitHub.

2. Sign in to your account.
3. Click New repository.
4. Enter the repository name as `SimpleInterest`.
5. Create the repository.
6. Copy the repository URL.

From the Maven project directory, initialize Git:

```
None
git init
git add .
git commit -m "Simple Interest Maven application"
git branch -M main
```

Connect the GitHub repository:

```
None
git remote add origin
https://github.com/YOUR_USERNAME/SimpleInterest.git
```

Push the project:

```
None
git push -u origin main
```

Replace `YOUR_USERNAME` with your GitHub username.

Step 9: Create the Azure Pipeline YAML file

Inside the Maven project root directory, create a file named:

```
None
nano azure-pipelines.yml
```

Enter the following pipeline configuration:

```
None

trigger:
  - main
```

```
pool:
  vmImage: 'ubuntu-latest'

steps:
  - task: JavaToolInstaller@0
    inputs:
      versionSpec: '17'
      jdkArchitectureOption: 'x64'
      jdkSourceOption: 'PreInstalled'

  - task: Maven@4
    inputs:
      mavenPomFile: 'pom.xml'
      goals: 'clean package'
      options: '-DskipTests'
      publishJUnitResults: false
      javaHomeOption: 'JDKVersion'
      jdkVersionOption: '1.17'
      mavenVersionOption: 'Default'

  - task: PublishBuildArtifacts@1
    inputs:
      PathtoPublish: '$(System.DefaultWorkingDirectory)/target'
      ArtifactName: 'SimpleInterest-JAR'
      publishLocation: 'Container'
```

Save and exit.

Explanation of the pipeline:

Pipeline section	Purpose
`trigger: main`	Starts the pipeline when changes are pushed to the main branch
`pool`	Uses an Ubuntu build agent

`JavaToolInstaller`	Configures Java 17
`Maven@4`	Executes Maven build commands
`clean package`	Compiles and packages the application
`PublishBuildArtifacts`	Publishes the generated JAR as a pipeline artifact

This pipeline compiles the source code and generates the JAR artifact.

Step 10: Push the pipeline file to GitHub

Execute:

```
None
git add azure-pipelines.yml
git commit -m "Add Azure pipeline"
git push
```

The GitHub repository should now contain:

```
None
SimpleInterest/
├── pom.xml
├── azure-pipelines.yml
└── src/
    └── main/
        └── java/
            └── com/example/App.java
```

Step 11: Create a pipeline in Azure DevOps

1. Open Azure DevOps.
2. Sign in using your Microsoft account.
3. Create a new organization if required.
4. Click New Project.

5. Enter the project name as `SimpleInterest`.
6. Set visibility to Private.
7. Click Create.

Step 12: Connect the GitHub repository

1. Open the Azure DevOps project.
2. Select Pipelines from the left sidebar.
3. Click Create Pipeline or New Pipeline.
4. Select GitHub as the source.
5. Authorize Azure DevOps to access your GitHub account if prompted.
6. Select the `SimpleInterest` repository.
7. Select Existing Azure Pipelines YAML file.
8. Choose the `main` branch.
9. Select `/azure-pipelines.yml`.
10. Click Continue.

Step 13: Run the Azure Pipeline

1. Review the YAML configuration.
2. Click Run.
3. Azure DevOps will allocate an Ubuntu build agent.
4. The pipeline will execute the following stages:
    - Configure Java 17.
    - Download Maven dependencies.
    - Compile the Java application.
    - Package the application into a JAR file.
    - Publish the JAR as a build artifact.

Wait until all pipeline tasks finish.

Expected result:

```
None
Build job: Succeeded
Maven: Succeeded
Publish Build Artifacts: Succeeded
```

The pipeline status should display Succeeded.

Step 14: Download and verify the JAR artifact

After the pipeline succeeds:

1. Open the completed pipeline run.
2. Select Summary.

3. Locate the Related or Artifacts section.
4. Click the artifact named `SimpleInterest-JAR`.
5. Download the artifact.
6. Verify that it contains:

```
None
SimpleInterest-1.0-SNAPSHOT.jar
```

The JAR can be executed using:

```
None
java -jar SimpleInterest-1.0-SNAPSHOT.jar
```

6. Final Result

Thus, a Java application to calculate simple interest was developed using Maven. The application was compiled and packaged into a JAR file, and an Azure Pipeline was configured to automatically build the application and publish the generated JAR artifact.

Important commands for the practical exam

Operation	Command
Check Java	`java -version`
Check Maven	`mvn -version`
Compile	`mvn compile`
Package JAR	`mvn clean package`
View JAR	`ls target`

Run JAR	`java -jar target/SimpleInterest-1.0-SNAPSHOT.jar`
Push to GitHub	`git push -u origin main`

Remember the workflow:

Java program → Maven `pom.xml` → Compile and package → JAR artifact → GitHub → Azure Pipeline → Build succeeded → Download artifact.

Question 2: Student Pass/Fail using JUnit, Maven and Azure Pipeline

1. Aim

To develop a Java application to determine whether a student has passed or failed based on the marks obtained, write JUnit test cases for different marks, and configure an Azure Pipeline using Maven to build the application and run regression tests.

2. Problem Statement

A student is considered:

- Pass if marks are greater than or equal to 50.
- Fail if marks are less than 50.

The application must be tested using the following marks:

Marks	Expected result
35	Fail
50	Pass

75	Pass
100	Pass

3. Requirements

- Java JDK 17
- Apache Maven
- Git and GitHub account
- Microsoft Azure DevOps account
- JUnit testing framework

4. Procedure

Step 1: Check Java and Maven installation

Open the terminal and execute:

```
None
java -version
```

```
None
mvn -version
```

If they are not installed on Ubuntu:

```
None
sudo apt update
sudo apt install openjdk-17-jdk maven -y
```

Verify the installation again.

Step 2: Create a Maven project

Create a project directory:

```
None
mkdir StudentResult
cd StudentResult
```

Generate the Maven project:

```
None
mvn archetype:generate \
-DgroupId=com.example \
-DartifactId=StudentResult \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

Navigate into the project:

```
None
cd StudentResult
```

The project structure will be:

```
None
StudentResult/
├── pom.xml
└── src/
    ├── main/
    │   └── java/
    │       └── com/example/App.java
    └── test/
        └── java/
            └── com/example/AppTest.java
```

Step 3: Develop the Java application

Open the Java source file:

```
None
nano src/main/java/com/example/App.java
```

Replace the code with:

None

```
package com.example;

import java.util.Scanner;

public class App {

    public static String checkResult(int marks) {
        if (marks >= 50) {
            return "Pass";
        } else {
            return "Fail";
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter student marks: ");
        int marks = sc.nextInt();

        if (marks < 0 || marks > 100) {
            System.out.println("Invalid marks");
        } else {
            String result = checkResult(marks);
            System.out.println("Student Result: " + result);
        }

        sc.close();
    }
}
```

Save the file using `Ctrl + O`, Enter, and `Ctrl + X`.

Explanation:

- `checkResult(int marks)` accepts the student's marks.
- If marks are greater than or equal to 50, it returns `"Pass"`.
- Otherwise, it returns `"Fail"`.
- The `main()` method accepts marks from the user and displays the result.

- The program also checks whether the entered marks are within the valid range of 0 to 100.

Step 4: Configure the Maven project

Open the `pom.xml` file:

```
None
nano pom.xml
```

Replace its contents with:

```
None

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>StudentResult</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding
>
        <junit.version>4.13.2</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive>
                        <manifest>

<mainClass>com.example.App</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

Save and exit.

Explanation:

- The JUnit dependency is used to write and execute unit tests.
- `scope=test` means JUnit is required only during testing.
- Maven Surefire Plugin executes the JUnit test cases.
- Maven Compiler Plugin compiles the Java program.

- Maven JAR Plugin packages the application into a JAR file.

Step 5: Create JUnit test cases

Open the test file:

```
None
nano src/test/java/com/example/AppTest.java
```

Replace the code with:

```
None

package com.example;

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AppTest {

    @Test
    public void testFailFor35Marks() {
        assertEquals("Fail", App.checkResult(35));
    }

    @Test
    public void testPassFor50Marks() {
        assertEquals("Pass", App.checkResult(50));
    }

    @Test
    public void testPassFor75Marks() {
        assertEquals("Pass", App.checkResult(75));
    }

    @Test
    public void testPassFor100Marks() {
        assertEquals("Pass", App.checkResult(100));
    }
}
```

Save and exit.

Explanation of JUnit test cases:

Test method	Input	Expected output
`testFailFor35Marks()`	35	Fail
`testPassFor50Marks()`	50	Pass
`testPassFor75Marks()`	75	Pass
`testPassFor100Marks()`	100	Pass

- `@Test` identifies a method as a test case.
- `assertEquals()` compares the expected result with the actual result.
- If the expected and actual values match, the test passes.
- If they differ, the test fails.

Step 6: Compile the application

From the project directory, execute:

```
None
mvn compile
```

Expected output:

```
None
BUILD SUCCESS
```

This confirms that the Java source code has compiled successfully.

Step 7: Run the application

Execute:

```
None
mvn exec:java -Dexec.mainClass="com.example.App"
```

If the exec plugin is not configured, package the application first and run the JAR as shown in Step 9.

Example:

```
None
Enter student marks: 75
Student Result: Pass
```

Another example:

```
None
Enter student marks: 35
Student Result: Fail
```

Step 8: Execute the JUnit regression tests

Run:

```
None
mvn test
```

Maven will:

1. Compile the main Java source code.
2. Compile the JUnit test source code.
3. Execute all test methods.
4. Compare actual results with expected results.
5. Generate a test report.

Expected output:

```
None
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS
```

The test report is generated at:

```
None
target/surefire-reports/
```

View the report:

```
None
cat target/surefire-reports/com.example.AppTest.txt
```

Regression testing: Regression testing verifies that the application continues to work correctly after changes are made to the source code. In this experiment, all four test cases are executed whenever the Maven test phase runs.

Step 9: Generate the JAR file

Execute:

```
None
mvn clean package
```

This command cleans previous build files, compiles the application, runs the tests, and packages the application.

Check the generated JAR:

```
None
ls target
```

Expected file:

```
None
StudentResult-1.0-SNAPSHOT.jar
```

Run the application:

```
None
java -jar target/StudentResult-1.0-SNAPSHOT.jar
```

The application will ask for marks and display the student's result.

5. Configure Azure Pipeline

Step 10: Create a GitHub repository

1. Open GitHub.
2. Sign in to your account.
3. Click New repository.
4. Enter the repository name as `StudentResult`.
5. Create the repository.
6. Copy the repository URL.

From the project directory, execute:

```
None
git init
git add .
git commit -m "Student pass fail application with JUnit"
git branch -M main
```

Connect the GitHub repository:

```
None
git remote add origin
https://github.com/YOUR_USERNAME/StudentResult.git
```

Push the files:

```
None
git push -u origin main
```

Replace `YOUR_USERNAME` with your GitHub username.

Step 11: Create the Azure Pipeline YAML file

In the project root directory, create:

```
None
nano azure-pipelines.yml
```

Enter the following YAML configuration:

```
None

trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

steps:
  - task: JavaToolInstaller@0
    inputs:
      versionSpec: '17'
      jdkArchitectureOption: 'x64'
      jdkSourceOption: 'PreInstalled'

  - task: Maven@4
    inputs:
      mavenPomFile: 'pom.xml'
      goals: 'clean package'
      publishJUnitResults: true
      testResultsFiles: '**/surefire-reports/TEST-*.xml'
      javaHomeOption: 'JDKVersion'
      jdkVersionOption: '1.17'
      mavenVersionOption: 'Default'

  - task: PublishBuildArtifacts@1
    inputs:
      PathtoPublish: '$(System.DefaultWorkingDirectory)/target'
      ArtifactName: 'StudentResult-JAR'
      publishLocation: 'Container'
```

Save and exit.

Explanation:

Pipeline instruction	Purpose
`trigger: main`	Runs the pipeline when changes are pushed to main
`pool`	Uses an Ubuntu build agent
`JavaToolInstaller`	Configures Java 17
`Maven@4`	Executes the Maven build
`clean package`	Compiles, tests, and packages the application
`publishJUnitResults`	Publishes the JUnit test results
`PublishBuildArtifacts`	Publishes the JAR artifact

Step 12: Push the pipeline file to GitHub

Execute:

```
None
git add azure-pipelines.yml
git commit -m "Add Azure pipeline for regression testing"
git push
```

Step 13: Create the pipeline in Azure DevOps

1. Open Azure DevOps.
2. Sign in and open your organization.
3. Create a new project named `StudentResult`.

4. Select Pipelines from the left sidebar.
5. Click Create Pipeline.
6. Select GitHub as the source.
7. Authorize Azure DevOps to access your GitHub account.
8. Select the `StudentResult` repository.
9. Choose Existing Azure Pipelines YAML file.
10. Select the `main` branch.
11. Select `/azure-pipelines.yml`.
12. Click Continue.

Step 14: Execute the Azure Pipeline

1. Review the YAML configuration.
2. Click Run.
3. Azure DevOps allocates an Ubuntu build agent.
4. The pipeline installs or configures Java 17.
5. Maven downloads the required dependencies.
6. The application is compiled.
7. All four JUnit test cases are executed.
8. The test results are published.
9. The JAR artifact is generated and published.

Expected successful output:

```
None
Build: Succeeded
Tests: 4 passed
Failures: 0
Errors: 0
Artifact: StudentResult-JAR
```

The pipeline status should display Succeeded.

Step 15: Download the JAR artifact

After the pipeline succeeds:

1. Open the completed pipeline run.
2. Select Summary.
3. Locate the Artifacts section.
4. Click `StudentResult-JAR`.
5. Download the artifact.
6. Verify that it contains:

```
None
StudentResult-1.0-SNAPSHOT.jar
```

6. Result

Thus, a Java application to determine whether a student has passed or failed was developed using Maven. JUnit test cases were written for marks 35, 50, 75, and 100. An Azure Pipeline was configured to compile the application, execute regression tests, and generate and publish the JAR artifact successfully.

Important commands for the practical exam

Operation	Command
Compile	`mvn compile`
Run JUnit tests	`mvn test`
Build and package	`mvn clean package`
View test report	`ls target/surefire-reports`
View JAR	`ls target`
Run JAR	`java -jar target/StudentResult-1.0-SNAPSHOT.jar`
Push code	`git push`

Remember the workflow: Java program → Maven configuration → JUnit test cases → Regression testing → GitHub → Azure Pipeline → Test results → JAR artifact.

Experiment 3: Gross Salary Calculation Using Jenkins

Aim

To develop a Java application to calculate the gross salary of an employee and use Jenkins to compile and execute the application.

Requirements

- Ubuntu/Linux system or cloud virtual machine
- Java JDK
- Jenkins
- Git and GitHub (optional, for source-code management)

Formula

$$GrossSalary = BasicSalary + HRA + DA$$

Where:

- Basic Salary – basic pay of the employee
- HRA – House Rent Allowance
- DA – Dearness Allowance

Step 1: Install and Verify Java

Open the terminal and execute:

```
None
sudo apt update
sudo apt install openjdk-17-jdk -y
```

Verify Java:

```
None
java -version
javac -version
```

If multiple Java versions are installed, select Java 17:

```
None
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Step 2: Install Jenkins

If Jenkins is already installed and the dashboard is accessible, skip this step.

Install Java and Jenkins on the Ubuntu cloud VM using the official Jenkins installation instructions. Ensure the VM's firewall/security group allows:

- 22 – SSH
- 8080 – Jenkins dashboard

Start and enable Jenkins:

```
None
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

Open Jenkins in the browser:

```
None
http://<VM-IP-ADDRESS>:8080
```

For a local installation:

```
None
http://localhost:8080
```

Step 3: Create the Java Application

Create a project directory:

```
None
mkdir GrossSalary
cd GrossSalary
```

Create the Java source file:

```
None
nano GrossSalary.java
```

Enter the following program:

```
None
public class GrossSalary {
    public static void main(String[] args) {
        if (args.length != 3) {
            System.out.println(
                "Usage: java GrossSalary <Basic> <HRA> <DA>"
            );
            return;
        }

        double basic = Double.parseDouble(args[0]);
        double hra = Double.parseDouble(args[1]);
        double da = Double.parseDouble(args[2]);

        double grossSalary = basic + hra + da;

        System.out.println("Basic Salary: " + basic);
        System.out.println("HRA: " + hra);
        System.out.println("DA: " + da);
        System.out.println("Gross Salary: " + grossSalary);
    }
}
```

Save the file.

Why command-line arguments are used: Jenkins runs builds non-interactively, so the salary values are passed as arguments instead of using `Scanner`.

Step 4: Test the Program Locally

Compile the program:

```
None
javac GrossSalary.java
```

Execute it with sample values:

```
None
java GrossSalary 25000 5000 3000
```

Expected output:

```
None
Basic Salary: 25000.0
HRA: 5000.0
DA: 3000.0
Gross Salary: 33000.0
```

Step 5: Configure Java in Jenkins

1. Open the Jenkins dashboard.
2. Navigate to Manage Jenkins → Tools (or Global Tool Configuration, depending on the Jenkins version).
3. Locate the JDK section.
4. Add a JDK installation if required.
5. Set the JDK name, for example:

```
None
```

6.
```
JDK17
```

7. If Java is already installed on the VM, configure its installation path, commonly:

```
None
```

8.
```
/usr/lib/jvm/java-17-openjdk-amd64
```

9. Save the configuration.

If the JDK section is unavailable in your Jenkins version, the build can use the system Java installed on the VM.

Step 6: Create a Jenkins Job

1. From the Jenkins dashboard, click New Item.
2. Enter the job name:

None

3.
```
GrossSalary-Job
```
4. Select Freestyle project.
5. Click OK.

Step 7: Configure Build Parameters

To provide salary values during each build:

1. Select This project is parameterized.
2. Add three String Parameter entries:

Name	Default value
BASIC	25000
HRA	5000
DA	3000

3. Under Build Steps, select Execute shell.
4. Enter:

None

```
echo "Compiling Java program..."
javac GrossSalary.java

echo "Executing Java program..."
java GrossSalary "$BASIC" "$HRA" "$DA"
```

5. Click Save.

Step 8: Provide the Source Code to Jenkins

Option A – Use a local source file

Place `GrossSalary.java` in the Jenkins job's workspace. For a quick practical demonstration, you can create the file directly in the workspace.

Option B – Use GitHub

1. Create a GitHub repository named `GrossSalary`.
2. Upload `GrossSalary.java`.
3. In Jenkins, open the job and click Configure.
4. Under Source Code Management, select Git.
5. Enter the repository URL.
6. Save the job.

Jenkins will check out the Java source code before executing the build step.

Step 9: Build and Execute the Application

1. Open `GrossSalary-Job`.
2. Click Build with Parameters.
3. Enter sample values:

```
None

BASIC = 25000
HRA   = 5000

   4. DA    = 3000
```

5. Click Build.
6. Open the build number.
7. Click Console Output.

Expected console output:

```
None
Compiling Java program...
Executing Java program...
Basic Salary: 25000.0
HRA: 5000.0
DA: 3000.0
```

```
Gross Salary: 33000.0
Finished: SUCCESS
```

Result

The Java application to calculate gross salary was developed and tested successfully. Jenkins was configured to compile and execute the application, and the gross salary was displayed in the build console.

Important: If your practical question expects HRA and DA to be calculated as percentages of basic salary, use the percentage formula specified in your lab manual instead.

Experiment 4: Palindrome Checking Using Jenkins CI and GitHub

Aim

To develop a Java application to check whether a given number is a palindrome and configure Jenkins for Continuous Integration (CI) using a GitHub repository.

Requirements

- Ubuntu/Linux system
- Java JDK
- Jenkins
- Git
- GitHub account

Step 1: Verify Java and Git

Open the terminal and execute:

```
None
java -version
javac -version
git --version
```

If Java or Git is not installed:

```
None
sudo apt update
sudo apt install openjdk-17-jdk git -y
```

Step 2: Create the Java Application

Create a project directory:

```
None
mkdir Palindrome
cd Palindrome
```

Create the Java file:

```
None
nano Palindrome.java
```

Enter the following code:

```
None
public class Palindrome {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Please provide a number.");
            return;
        }

        int number = Integer.parseInt(args[0]);
        int original = number;
        int reverse = 0;

        while (number > 0) {
            int digit = number % 10;
            reverse = reverse * 10 + digit;
            number = number / 10;
        }

        if (original == reverse) {
            System.out.println(original + " is a palindrome.");
```

```
        } else {
            System.out.println(original + " is not a
palindrome.");
        }
    }
}
```

Save and exit.

Step 3: Test the Program Locally

Compile the program:

```
None
javac Palindrome.java
```

Run it with a palindrome number:

```
None
java Palindrome 121
```

Expected output:

```
None
121 is a palindrome.
```

Test with a non-palindrome:

```
None
java Palindrome 123
```

Expected output:

```
None
123 is not a palindrome.
```

Step 4: Create a GitHub Repository

1. Open GitHub.
2. Create a new repository named `Palindrome`.
3. Keep it public or configure Jenkins credentials if it is private.
4. In the project directory, execute:

```
None
git init
git add Palindrome.java
git commit -m "Add palindrome Java program"
git branch -M main
git remote add origin
https://github.com/<username>/Palindrome.git
git push -u origin main
```

Replace `<username>` with your GitHub username.

Step 5: Configure Jenkins

1. Open Jenkins:

```
None
```

2. `http://localhost:8080`
3. For a cloud VM, use:

```
None
```

4. `http://<VM-IP-ADDRESS>:8080`
5. From the dashboard, click New Item.
6. Enter the job name:

```
None
```

7. `Palindrome-CI`
8. Select Freestyle project.
9. Click OK.

Step 6: Configure GitHub Source Code Management

1. Open the job's Configure page.
2. Under Source Code Management, select Git.
3. Enter the repository URL:

```
None
```

4.
```
https://github.com/<username>/Palindrome.git
```

5. Set the branch specifier to:

```
None
```

6.
```
*/main
```

7. If the repository is private, add the required GitHub credentials.
8. Under Build Steps, select Execute shell.
9. Enter the following commands:

```
None
echo "Compiling Java program..."
javac Palindrome.java

echo "Checking palindrome..."
java Palindrome 121
```

8. Click Save.

Step 7: Run the Jenkins Job

1. Open the `Palindrome-CI` job.
2. Click Build Now.
3. Select the latest build number.
4. Click Console Output.

Expected output:

```
None
Compiling Java program...
Checking palindrome...
121 is a palindrome.
Finished: SUCCESS
```

Step 8: Configure Continuous Integration

To trigger Jenkins automatically whenever code is pushed to GitHub:

1. Open the Jenkins job and select Configure.
2. Under Build Triggers, enable GitHub hook trigger for GITScm polling.
3. Save the job.
4. In GitHub, open the repository's Settings → Webhooks.
5. Click Add webhook.
6. Enter the Jenkins URL followed by:

```
None
```

7. `/github-webhook/`
8. Example:

```
None
```

9. `http://<VM-IP-ADDRESS>:8080/github-webhook/`
10. Set the content type to `application/json`.
11. Select Just the push event.
12. Add the webhook.

Note: For a cloud VM, Jenkins must be reachable from GitHub for webhook-triggered builds. Use an appropriately secured, reachable Jenkins endpoint.

Step 9: Verify CI

Make a change to `Palindrome.java`, then push it to GitHub:

```
None
git add Palindrome.java
git commit -m "Update palindrome program"
git push
```

Jenkins should automatically start a new build. Check the build history and console output to verify that the latest source code was compiled and executed.

Result

The Java palindrome application was developed and tested successfully. Jenkins was integrated with GitHub to compile and execute the application, and Continuous Integration was configured to trigger builds on code pushes.

Experiment 5: Deploy a Web Application Using Ansible

Aim

To deploy a web application on an Apache web server using Ansible.

Requirements

- Ubuntu/Linux system
- Ansible
- Apache web server
- Web application HTML file

Step 1: Install Ansible

Open the terminal and execute:

```
None
sudo apt update
sudo apt install ansible -y
```

Verify installation:

```
None
ansible --version
```

Step 2: Create the Project Directory

```
None
mkdir AnsibleWebApp
cd AnsibleWebApp
```

Step 3: Create the Web Application

Create an HTML file:

```
None
nano index.html
```

Enter:

```
None
<!DOCTYPE html>
<html>
<head>
    <title>Ansible Web Application</title>
</head>
<body>
    <h1>Web Application Deployed Successfully!</h1>
    <p>This application was deployed using Ansible.</p>
</body>
</html>
```

Save and exit.

Step 4: Create the Ansible Inventory

Create an inventory file:

```
None
nano inventory
```

For deploying on the same Ubuntu machine, enter:

```
None
[webservers]
localhost ansible_connection=local
```

Save the file.

Step 5: Create the Ansible Playbook

Create the playbook:

```
None
nano webapp.yml
```

Enter:

```
None
---
- name: Deploy Web Application
  hosts: webservers
  become: yes

  tasks:
    - name: Install Apache web server
      apt:
        name: apache2
        state: present
        update_cache: yes

    - name: Start and enable Apache
      service:
        name: apache2
        state: started
        enabled: yes

    - name: Deploy the web application
      copy:
        src: index.html
        dest: /var/www/html/index.html
        mode: '0644'

    - name: Verify the web application
      uri:
        url: http://localhost
        status_code: 200
```

Step 6: Execute the Ansible Playbook

Run the playbook:

```
None
ansible-playbook -i inventory webapp.yml --ask-become-pass
```

Enter the sudo password when prompted.

If using the inventory format `localhost ansible_connection=local`, Ansible will execute the tasks on the local machine.

Step 7: Verify Deployment

Open a browser and enter:

```
None
http://localhost
```

The webpage should display:

Web Application Deployed Successfully!

You can also verify Apache status:

```
None
sudo systemctl status apache2
```

Step 8: Verify Using Ansible

Run:

```
None
ansible webservers -i inventory -m ping
```

Expected output should indicate:

```
None
"ping": "pong"
```

Result

Ansible was installed and configured successfully. The playbook installed Apache, started the service, deployed the HTML web application, and verified that the application was accessible.

Experiment 6: Vowel or Consonant Checker Using Gradle

Aim

To develop a Java application that determines whether an entered alphabet is a vowel or consonant and build the application using Gradle.

Requirements

- Ubuntu/Linux system
- Java JDK
- Gradle
- Text editor or IDE

Step 1: Verify Java and Gradle

```
None
java -version
javac -version
gradle -version
```

If Java is not installed:

```
None
sudo apt update
sudo apt install openjdk-17-jdk -y
```

If Gradle is not installed:

```
None
sudo apt install gradle -y
```

Step 2: Create a Gradle Project

Create the project directory:

```
None
mkdir VowelConsonant
cd VowelConsonant
```

Create the Gradle project structure:

```
None
mkdir -p src/main/java
```

The directory structure will be:

```
None
VowelConsonant/
├── build.gradle
└── src/
    └── main/
        └── java/
            └── Main.java
```

Step 3: Create the Java Program

Create the source file:

```
None
nano src/main/java/Main.java
```

Enter the following code:

```
None
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an alphabet: ");
        char ch = sc.next().charAt(0);
```

```
        ch = Character.toLowerCase(ch);

        if (ch < 'a' || ch > 'z') {
            System.out.println("Invalid input. Enter an
alphabet.");
        } else if (ch == 'a' || ch == 'e' || ch == 'i'
                || ch == 'o' || ch == 'u') {
            System.out.println(ch + " is a vowel.");
        } else {
            System.out.println(ch + " is a consonant.");
        }

        sc.close();
    }
}
```

Step 4: Create the Gradle Build File

Create `build.gradle` in the project directory:

```
None
nano build.gradle
```

Enter:

```
None
plugins {
    id 'application'
}

application {
    mainClass = 'Main'
}
```

Save and exit.

Step 5: Build the Application

From the project directory, execute:

```
None
gradle build
```

Gradle compiles the Java source code and creates the build output.

Expected result:

```
None
BUILD SUCCESSFUL
```

Step 6: Execute the Application

Run:

```
None
gradle run
```

Example output:

```
None
Enter an alphabet: a
a is a vowel.
```

Another example:

```
None
Enter an alphabet: b
b is a consonant.
```

Step 7: Generate the JAR File

Execute:

```
None
gradle jar
```

The JAR file will be generated in:

```
None
build/libs/
```

To view it:

```
None
ls build/libs
```

Result

The Java application to identify vowels and consonants was developed successfully. Gradle was used to compile, build, execute, and package the application into a JAR file.